UNIVERSITY OF WOLLONGONG IN DUBAI

UNIVERSITY OF WOLLONGONG IN DUBAI

CSCI316 Cover Sheet

Subject Code:	**CSCI316**
Subject Name:	**Big Data Mining Implementations & Techniques**
Submission Type:	**Project 2**
Project Title:	**CSCI316 Report 2**
Student/Team Name:	**Mohammed Atwani**
Student E-mail:	Ma611@uowmail.edu.au
Lecturer Name:	**Dr. Patrick Mukala**
Due Date:	21/03/2025
Date Submitted:	20/10/2025

PLAGIARISM: The penalty for deliberate plagiarism is FAILURE in the subject. Plagiarism is cheating by using the written ideas or submitted work of someone else. UOWD has a strong policy against plagiarism. The University of Wollongong in Dubai also endorses a policy of nondiscriminatory language practice and presentation. **PLEASE NOTE:** STUDENTS MUST RETAIN A COPY OF ANY WORK SUBMITTED	**DECLARATION:** I/We certify that this is entirely my/our own work, except where I/we have given fully-documented references to the work of others, and that the material contained in this document has not previously been submitted for assessment in any formal course of study. I/we understand the definition and consequences of plagiarism. We/I declare that the project proposal has not been used in any UOWD courses before and that this project idea is a total new idea of the project team **Signature of Student: Mohammed Atwani**

Optional Marks:

Comments:

Lecturer Project Proposal Receipt (To be filled in by student and retained by Lecturer upon return of assignment)

Subject: CSCI316 **Project Title:** CSCI316 Project 2

Due Date: 21/03/2025 **Date Submitted:** 20/03/2025

Signature of Student: Mohammed Atwani

UNIVERSITY
OF WOLLONGONG
IN DUBAI

Student Project

Proposal Receipt (To be filled in and retained by Student upon submission of assignment)

Subject:

Project Title:

Student/Team Name:

Student Number:

Due Date:

Date Submitted:

Signature of Lecturer:

UNIVERSITY OF WOLLONGONG IN DUBAI

CSCI316 REPORT

Name	Student ID
Mohammed Atwani	8485811
Omar Massoud	7858681
Moustafa Hegazi	8194166
Elyas Sayed	8479574
Fares Khater	8250704

UNIVERSITY OF WOLLONGONG IN DUBAI

Table of Contents

Presentation Link: **5**

https://youtu.be/14a2IQUC8IY **5**

Github Link: **5**

1. Introduction & Problem Statement **5**

1.1 Background of NLP and LLMs 5

1.2 Problem of Low-Resource Languages 5

1.3 Importance of Transfer Learning 5

1.4 Project Objective and Overview 6

2. Task & Language Justification **6**

2.1 Selected Language and Task (Arabic & Arabizi Sentiment Analysis) 6

2.2 Why Transfer Learning is Necessary 6

2.3 Challenges and Potential Failure Modes 7

3. Data Description **7**

3.1 Dataset Sources and Structure (Arabic Tweets & Arabizi CSVs) 7

3.2 Data Challenges (Noise, Code-Switching, Imbalance) 7

3.3 Preprocessing Techniques 8

4. Methodology **8**

4.1 Model Selection (BERT / mBERT Justification) 8

4.2 Transfer Learning Approaches (Full FT vs LoRA) 9

4.3 Framework Implementation (PyTorch vs TensorFlow) 9

5. Experiments & Results **9**

5.1 Experimental Setup (Split, Hyperparameters, Hardware) 9

5.2 Quantitative Results (Accuracy, F1, Comparison Tables) 10

5.3 Qualitative Analysis (Examples & Errors) 11

6 . Cultural Evaluation **11**

6.1 Proposed Evaluation Metric 11

6.2 Evaluation Methodology 11

6.3 Results and Interpretation 12

7 . Discussion **13**

7.1 Key Findings 13

7.2 Limitations and Challenges 13

7.3 Transfer Learning Insights 13

8 . Conclusion & Future Work **14**

8.1 Summary of Results 14

8.2 Key Takeaways 14

8.3 Future Improvements 14

9. References **14**

UNIVERSITY OF WOLLONGONG IN DUBAI

Presentation Link:

https://youtu.be/14a2IQUC8IY

Github Link:

https://github.com/AtwaniGG/CSCI316-Project-2

1. Introduction & Problem Statement

1.1 Background of NLP and LLMs

Natural Language Processing (NLP) is a branch of artificial intelligence that aims at equipping machines to read, comprehend and create human languages. Over the past few years, Large Language Models (LLMs) have become highly efficient in a variety of NLP tasks, including text classification, translation, sentiment analysis, and so on. The models are constructed based on transformer frameworks that are based on attention-based mechanisms that use the contextual relationships among words within a sentence.

LLMs are normally trained on huge datasets to enable them to acquire general linguistic regularities and semantic articulations. This pre-training step allows the models to be fine-tuned to fit certain downstream tasks and therefore is highly useful in various applications in the real world.

1.2 Problem of Low-Resource Languages

Regardless of the success of LLMs, their output is extremely sensitive to the quality and size of large datasets. Most languages such as Arabic dialects and the Romanized Arabic Arabizi are low-resource languages since they do not have standard datasets and linguistic variation. Arabizi also has other issues, being composed in Latin letters and figures to signify Arabic sounds (e.g., the letter 7 means the sound [ح]) and does not have any regular rules of spelling. Moreover, text messages have a lot of slangs, abbreviations, emojis, and switching between Arabic and English, which makes it even harder to be processed by models.

Consequently, the performance of traditional machine learning solutions on this kind of data is not very strong, which justifies the need to adopt more significant methods.

1.3 Importance of Transfer Learning

Transfer learning helps with the low-resource language constraint since it uses the knowledge of the already trained models on big datasets. Unlike training a model with raw data, a transfer first requires a large amount of data and computation time, transfer learning can be used to fine-tune an existing trained model on a specific task with a smaller amount of data.

In the project, transfer learning allows the adaptation of a BERT-based model, which is trained on large multilingual or English datasets, to sentiment analysis of Arabic and Arabizi text.

Transference of learned representations of syntax, semantics, and contextual knowledge are transferred to the model making it perform better with limited training data.
This method is especially significant with those languages in which annotated data is both rare and inconsistent such as Arabizi.

1.4 Project Objective and Overview

This project aims at establishing a transfer learning model of sentiment analysis on Arabic and Arabizi data. The model will be used to categorize text into a sentiment, that is, positive, negative, and neutral.
The project implies a preprocessing of multilingual and noisy text data, choosing a suitable pre-trained model (BERT-based) and two transfer learning methods, including full fine-tuning and parameter-efficient fine-tuning. Moreover, both PyTorch and TensorFlow frameworks are used to implement it to compare the performance and flexibility.
The aim of the whole research is to analyze the effectiveness of transfer learning in adapting the pre-trained models to low-resource language tasks, especially when using social media data.

2. Task & Language Justification

2.1 Selected Language and Task (Arabic & Arabizi Sentiment Analysis)

This project is aimed at sentiment analysis of text that is in Modern Standard Arabic (MSA) and Arabizi. They are well spread language forms used in the social media platforms in the MENA region, especially in informal communication.
Sentiment analysis refers to the process of identifying the tone of a text in terms of emotion. The sentiment in the model in this project involves categorizing input text under positive, negative and neutral sentiment. The given task is extremely applicable in terms of its application in customer feedback analysis, opinion mining, and social media monitoring applications.
The fact that the Arabizi is not standardized and has mixed linguistic characteristics also increases the complexity of the task since it is a perfect opportunity to test the transfer learning methods.

2.2 Why Transfer Learning is Necessary

This project requires transfer learning because only a small amount of big-scale and labeled datasets of Arabic dialects and Arabizi are available. It would take considerably more data and computing resources that are not available to train a deep learning model.
Through a trained BERT model the project would use the already acquired linguistic knowledge by way of word embeddings, contextual relationship and sentence structure. The model retains this information in the layers of the transformer and attention.
Fine-tuning enables the model to adapt this knowledge to the task of sentiment analysis and enhances the model, thereby reducing training time and resource needs.

2.3 Challenges and Potential Failure Modes

There are a number of challenges that are presented in implementing transfer learning to Arabic and Arabizi sentiment analysis.
The inaccuracy of Arabizi spelling, i.e. the same word is spelled in various forms is one of the biggest issues. This may cause trouble with tokenization and a decrease in model accuracy. Also, the possibility of code-switching between Arabic and English sentences in one sentence may mislead the model to manipulate its predictions.

The second weakness is that there is a possible cultural difference between the training information of the pre-trained models and the target language. Not all models have been trained on Arabic text, so they might not be able to pick up culture and context-specific meaning within Arabic text.

Lastly, social media data may contain noise, which may be emojis, abbreviations, and informal expressions that result in ambiguity and deteriorate the classification performance.

3. Data Description

3.1 Dataset Sources and Structure (Arabic Tweets & Arabizi CSVs)

This task utilizes publicly available, text-based data that represents actual Arabic and Arabic-English social media environments.

The data used in this task is as follows:

The Arabic Tweets dataset is available in TSV and contains user tweets with positive or negative labels, representing informal and dialectal Arabic for real-world sentiment analysis.

The Arabizi dataset is available in CSV and contains Latin script Arabic with both binary and multi-class sentiment labels. The dataset is divided into train, validation, and test sets, reflecting the informal and misspelled nature of arabic text.

code-switching refers to Arabic and English words being used together in sentences, which is common in UAE society due to their mixed community.

All datasets contain standardized text and sentiment labels, allowing for uniform preprocessing and training.

3.2 Data Challenges (Noise, Code-Switching, Imbalance)

The data set for the study also has some challenges, include the following social media text challenges: linguistic noise, including misspellings, grammatical errors, the use of emojis, hashtags, and the use of stretched or repeated characters; and code-switching, which includes the switching of two or more languages in the sentence, creating both syntactic and semantic complexity, which makes it difficult for the model to maintain the context.

The Arabizi script also has some challenges, include the fact that it has non-standard spellings, whereby some Arabic words may be spelled in various ways, including the use of combinations of digits, creating vocabulary noise, as well as the fact that some data sets may be unbalanced, with some having more positive or neutral sentiments than others.

3.3 Preprocessing Techniques

In order to deal with noisy and multi-linguistic data, several preprocessing steps are employed. The first step involves data cleaning, where clutter such as URLs, user mentions, and hashtags are removed to concentrate on the linguistic data.

In Arabic text, normalization is done by standardizing the characters and removing diacritical marks, thus reducing sparsity and handling different styles and dialects.

In the case of Arabizi, sometimes it is used as is to learn patterns, while sometimes it is translated into Arabic script based on the experiment.

In tokenization, the model's tokenizer such as BERT, is used to incorporate sub-word information, which is important in Arabic due its morphology.

In handling class imbalance, class-weighted loss is employed. The data is split into training, validation, and test sets with a 70-15-15 ratio.

4. Methodology

4.1 Model Selection (BERT / mBERT Justification)

In this study, the transformer-based models are used as the backbone of the sentiment analysis of the code-switched text. BERT and mBERT were selected as they have been shown to have excellent contextual representation capabilities.

For the monolingual baselines, AraBERT and CAMeLBERT were selected, as they have been shown to have excellent performance on monolingual Arabic text due to their pre-training on vast amounts of Arabic data. However, they have been shown to perform poorly on code-switching.

On the other hand, mBERT has been pre-trained on over 100 languages, including Arabic and English, making it an ideal candidate for the task of code-switching and sentiment analysis, as it has the capability of bridging the linguistic gaps between languages while maintaining efficiency.

4.2 Transfer Learning Approaches (Full FT vs LoRA)

There are two transfer learning methods that can be used to adapt pre-trained models to the target task. The first one is full fine-tuning, in which all parameters are updated. The second one is parameter-efficient fine-tuning using LoRa (Low-Ranked Adaptation), in which extra metrics are added, but the parameters are not updated.

The study compares the two models in terms of performance, efficiency, and resource consumption, with the goal being to check whether parameter-efficient fine-tuning is as effective as full fine-tuning but with less resource consumption.

4.3 Framework Implementation (PyTorch vs TensorFlow)

To ensure that there is a comprehensive understanding of the modern deep learning frameworks, the implementation is done using both PyTorch and TenserFlow. PyTorch, in conjunction with the huggingFace transformers library, is used for the main implementation. This is because PyTorch, with its dynamic computation graph and flexibility, is ideal for rapid development and experimentation. Moreover , it allows for the use of advanced features such as LoRA using other libraries. TenserFlow is used in conjunction with the Keras API to implement the same model. TenserFlow is a traditional deep learning framework that uses static graphs. However, with the use of eager execution, TenserFlow is more intuitive in model development.

The implementation of the same model using two different deep learning frameworks provides a comparative analysis of the two frameworks. This is a significant contribution of this project. Moreover, it also showcases an understanding of the implementation of transfer learning in different environments.

5. Experiments & Results

5.1 Experimental Setup (Split, Hyperparameters, Hardware)

A collection called Raïdy held nearly eighteen hundred texts, evenly spread across three sentiment types six hundred each. Splitting followed a method that kept proportions steady: training got 1,679, validation 359, and testing 360. Another set, much larger, came in Arabic with fifty-six thousand messages marked just two ways, going into early training under PyTorch before touching the main task.

Running mBERT bert-base-multilingual-cased, packing 178 million parameters used HuggingFace Transformers inside Google Colab with GPU via CUDA. Batch processing handled 16 samples at once, sequences capped at 128 tokens. Training ran up to three passes, halting sooner if improvements stalled across two rounds. Full fine-tuning used a learning pace of 0.00002, while LoRA shifted that to 0.0003, operating with a rank of eight balanced against a scaling factor of sixteen. The optimizer differed slightly too under PyTorch, Adam under TensorFlow, both pushing against a cross-entropy loss.

Arabizi preprocessing mapped digit-letters to Arabic Unicode — '3'→'ح'→'7' ,'ع' — dropping the out-of-vocabulary rate from 0.92% down to 0.01%, nearly wiping out unknown tokens entirely.

5.2 Quantitative Results (Accuracy, F1, Comparison Tables)

Model / Strategy	Macro F1	Accuracy	Trainable Params	Train Time	Framework
No-Transfer Baseline	0.22	42%	178M (100%)	137s	PyTorch
Full Fine-Tuning (Transfer)	0.63	68%	178M (100%)	138s	PyTorch
LoRA / PEFT (Transfer) ★	0.65	68%	297K (0.17%)	83s	PyTorch
Full Fine-Tuning	0.66	69.4%	178M (100%)	205s	TF/Keras
LoRA / PEFT	0.23	50.1%	~2K (0.00%)	103s	TF/Keras

Table 5.1 — All model results. ★ = best efficiency-adjusted result. TF/Keras LoRA collapsed to majority-class prediction.

Midway through the results, one pattern stands out: when it comes to per-class F1, the Neutral category - making up half the dataset - holds steady between 0.77 and 0.80 no matter which transferred model runs. On the flip side, Negative and Positive hover lower, anywhere from 0.48 to 0.67. Without any transfer at all, performance drops hard; especially for underrepresented labels, where F1 sinks to about 0.04. That sharp fall shows why borrowing signals across languages matters. Among methods tested, PyTorch LoRA strikes a rare mix - accuracy like full fine-tuning, yet finishes quicker by two-fifths, using just a fraction of the weights.

5.3 Qualitative Analysis (Examples & Errors)

Sentiment Ambiguity: Mistakes often popped up when words carried mixed feelings - like "I miss you a lot, Amar" tagged right (Positive ✓), yet "see you tomorrow at work" marked off despite being negative (Negative ✓). Confusion usually stemmed from unclear emotional hints in phrases.

Sarcasm and Subtlety: Sure thing looks good on paper, yet hides a problem underneath - systems get it wrong when sarcasm wears a smile. This happens because there just isn't enough clear data to learn from, especially where resources are thin. What seems friendly at first glance ends up twisted in meaning, but models miss the turn. Training falls short, not by accident, simply by lack of example. The gap stays wide where subtle language plays hide and seek.

Code-switching fragmentation: Tweets blending Arabic, Latin, French (e.g., 'crop top', 'por la decima'), and digit-letters produce fragmented tokens (e.g., 'saعٔa') that misalign with mBERT's multilingual embeddings.

Neutral Class Over-assignment: That Full FT model grabs most neutral tweets - hits 75% or higher on those - but stumbles on negative ones, only tagging 38%. It tends to sweep unclear posts into neutral ground, almost like it's hoarding uncertain cases instead of deciding. Ambiguous content often ends up filed there, simply because the system leans that way when unsure.

Colloquialism Mismatch: When it comes to Lebanese speech, some everyday phrases like 'yu2brnee' or 'N2ebre lbese' don't exist in formal Arabic at all. Because of that, leaning on standard forms doesn't help much here. These words live only in casual talk, nowhere else. Trying to match them with textbook versions falls flat every time.

6 . Cultural Evaluation

6.1 Proposed Evaluation Metric

A score like accuracy won't show if performance dips on certain language forms. Though some tweets mix scripts, others switch completely differences that expose hidden slant in results. When a system falters only on highly romanized samples, overall numbers might still look fine, yet that gap points to uneven understanding across variations people actually use.

Arabizi intensity tracks this by counting Latin letters divided by all alphabetic characters in a message zero means only Arabic script, one means fully Latin. Test examples are split into two halves by this score; macro F1 is computed separately for each, and the gap between them formally |F1_High − F1_Low| becomes the Dialectal Transfer Gap Score. Smaller numbers suggest script-agnostic stability, while larger gaps reveal weaknesses tied to specific user patterns that overall scores would otherwise hide.

6.2 Evaluation Methodology

Every test sample had its intensity measured by latin_char_ratio (ASCII letters divided by all letter characters). Notably, two peaks showed up in the data - most people stuck to just one script per message. Because of that pattern, both PyTorch and TensorFlow setups sorted results into Low versus High categories, 180 samples each. F1 scores were calculated separately for these groups, relying on scikit-learn tools. All five trained models then went through DTGS to allow clear comparisons across strategies.

6.3 Results and Interpretation

Model	Model	F1(low)	F1 (High)	DTGS	Interpretation
No-Transfer Baseline	PyTorch	0.1698	0.2721	0.1023	Worst — largest intensity gap
Full Fine Tuning	PyTorch	0.5639	0.6308	0.0668 ★	Best robustness among transfer models
LoRA	PyTorch	0.5691	0.6401	0.0710	Comparable; slightly wider gap
Full Fine Tuning	TF/KERAS	0.5577	0.6879	0.1302	Higher gap than PyTorch FT
LoRA	TF/KERAS	0.1912	0.2645	0.0733	Low DTGS but collapsed model — misleading

Table 6.1 — DTGS results. ★ = most robust transfer model. TF/Keras uses variance; PyTorch uses absolute gap.

Out of the DTGS review, one thing stands clear right away: transfer learning narrows the dialectal gap. Without it, scores shift widely by intensity, starting at 0.1023 — switching to full fine-tuning or LoRA brings that down to 0.067–0.071. Moving from MSA to Arabizi helps stability, not only boosting precision but also making performance vary less across intensity levels.

There is another point that follows close behind: every model handles fully-romanised Arabizi more effectively, even though mBERT never saw romanised Arabic during training. The reason hides in what people say intense Roman script posts often carry clearer feelings, while messages mixing scripts become harder to pin down, full of unclear meanings.

Last comes a detail few might expect. Full fine-tuning edges out LoRA by a hair in stability; its DTGS sits at 0.0668 compared to 0.0710. On the other side, speed and size swing hard toward LoRA, it trains nearly half again as fast while using just a fraction of the parameters. Real-world use tends to pull things toward LoRA those gains often matter more than tiny stability bumps.

When you check DTGS, always look at the full F1 score too. The TF/Keras LoRA version shows just 0.0733 variance, yet it fails by guessing the most common class every time. Small DTGS gaps mean nothing if the model barely works anywhere.

DTGS shows what macro F1 hides, every model struggles more on mixed-script text than fully-romanised Arabizi, no matter the approach. To shrink that difference, someone might try focused data expansion or pull samples by intensity level, especially where usage dips lowest.

7 . Discussion

7.1 Key Findings

The findings have clearly demonstrated that transfer learning is effective in enhancing the sentiment classification effectiveness as opposed to the base model. The no-transfer model attained the lowest accuracy at 42% whereas the transfer learning models scored the highest accuracy of 69.4 and F1 score of 0.66 as shown in the results table.

PyTorch LoRA had the most favorable efficiency-performance trade-off, with comparable accuracy and only a small fraction of the parameters, and a smaller training time. Also, the transfer learning enhanced performance in all the sentiment classes, especially the underrepresented labels.

7.2 Limitations and Challenges

Although these were improved, a number of challenges influenced performance. Arabizi text was highly variable because of inconsistent spelling and through the use of numeric characters, tokenization was challenging. On the other hand, the code-switching between Arabic and English lowered model accuracy through the introduction of mixed linguistic structures.

Moreover, the qualitative analysis showed some problems in the areas of sarcasm, ambiguity and informal words that were difficult to decipher by the model. The model was also over-predicting the neutral class due to dataset imbalance, which lowered the performance on minorities.

7.3 Transfer Learning Insights

The results provide evidence that transfer learning is quite effective with low-resource language tasks. It allows using pre-trained linguistic knowledge to make models perform well with little data. Nevertheless, the findings also emphasize that model performance is also pegged on the

quality of the dataset and language consistency especially when dealing with noisy and code-switched text.

8 . Conclusion & Future Work

8.1 Summary of Results

The project managed to integrate the concept of transfer learning into sentiment analysis of Arabic and Arabizi text. BERT-based models were much more effective than the baseline, and the high scores reached almost 70 percent. The simulation of the PyTorch and TensorFlow versions showed also that the two systems can give credible results, although the performance of the former was better in parameter-saving tuning.

8.2 Key Takeaways

Transfer learning offers a good solution to addressing low-resource languages because it has fewer data requirements but good performance. Nonetheless, variability of the Arabizi, code-switching and text noise in social media are some of the challenges that continue to affect the accuracy of the model..

8.3 Future Improvements

Future research might aim at enhancing preprocessing methods especially in normalizing Arabizi text and mixed language inputs. Moreover, bigger datasets and Arabic-specific language models may also be used and increase performance. More complicated techniques of sarcasm-detection and contextual meaning-detection can also enhance the accuracy of classification.

9. References

Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT 2019*, 4171–4186. https://arxiv.org/abs/1810.04805

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. *Proceedings of ICLR 2022*. https://arxiv.org/abs/2106.09685

Raïdy, M., & Harmanani, H. (2023). A deep learning approach for sentiment and emotional analysis of Lebanese Arabizi Twitter data. *ITNG 2023: 20th International Conference on Information Technology-New Generations*, 27–35.

Saad, M. (2020). *Arabic sentiment Twitter corpus* [Dataset]. Kaggle. https://www.kaggle.com/datasets/mksaad/arabic-sentiment-twitter-corpus

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., & Brew, J. (2020). Transformers: State-of-the-art natural language processing. *Proceedings of EMNLP 2020: System Demonstrations*, 38–45. https://arxiv.org/abs/1910.03771

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., & Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32. https://arxiv.org/abs/1912.01703

10. Appendix

DEMO Code with output:

```python
BASE = '/content/drive/MyDrive/NLP/Arabizi Tweets'
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

tokenizer = AutoTokenizer.from_pretrained(f'{BASE}/models', local_files_only=True)
model = AutoModelForSequenceClassification.from_pretrained(
    f'{BASE}/models',   # ← directly here, no /full_ft
    num_labels=3,
    local_files_only=True
).to(device)
model.eval()
print("✓ Model loaded")


# ── Prediction function ──────────────────────────────────────────
LABELS = {0: '🔴 Negative', 1: '⚪ Neutral', 2: '🟢 Positive'}

def predict_sentiment(text):
    cleaned = clean_arabizi(text)
    inputs = tokenizer(
        cleaned, return_tensors='pt',
        truncation=True, max_length=128, padding=True
    ).to(device)
    with torch.no_grad():
        logits = model(**inputs).logits
    probs = torch.softmax(logits, dim=-1)[0].cpu().tolist()
    pred = logits.argmax(-1).item()

    print(f"\n{'-'*50}")
    print(f"  Input:      {text}")
    print(f"  Cleaned:    {cleaned}")
    print(f"  Prediction: {LABELS[pred]}")
    print(f"  Confidence: {max(probs)*100:.1f}%")
    print(f"  Scores:     Neg={probs[0]:.2f} | Neu={probs[1]:.2f} | Pos={probs[2]:.2f}")
    print(f"{'-'*50}")

# ── Run demo on sample inputs ────────────────────────────────────
demo_texts = [
    "hatha ktir 7elo ana mabsout ktir",       # positive Arabizi
    "shu hal hmar hayda ma biy3ref shi",       # negative Arabizi
    "rouhna 3al mall wshefna shweyyet nas",    # neutral Arabizi
    "هذا المنتج رائع جداً وأنا سعيد",          # positive MSA Arabic
    "I love this w ana mabsout",               # mixed English-Arabizi
]

for text in demo_texts:
    predict_sentiment(text)
```

```
Loading weights: 100%|▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰▰| 201/201 [00:00<00:00, 521.55it/s, Materializing param=classifier.weight]
✓ Model loaded

--------------------------------------------------
  Input:      hatha ktir 7elo ana mabsout ktir
  Cleaned:    hatha ktir ح7elo ana mabsout ktir
  Prediction: 🟢 Positive
  Confidence: 66.7%
  Scores:     Neg=0.23 | Neu=0.11 | Pos=0.67
--------------------------------------------------

--------------------------------------------------
  Input:      shu hal hmar hayda ma biy3ref shi
  Cleaned:    shu hal hmar hayda ma biyعref shi
  Prediction: 🔴 Negative
  Confidence: 55.0%
  Scores:     Neg=0.55 | Neu=0.19 | Pos=0.26
--------------------------------------------------

--------------------------------------------------
  Input:      rouhna 3al mall wshefna shweyyet nas
  Cleaned:    rouhna عal mall wshefna shweyyet nas
  Prediction: ⚪ Neutral
  Confidence: 47.4%
  Scores:     Neg=0.41 | Neu=0.47 | Pos=0.11
--------------------------------------------------

--------------------------------------------------
  Input:      هذا المنتج رائع جداً وأنا سعيد
  Cleaned:    هذا المنتج رائع جداً وأنا سعيد
  Prediction: ⚪ Neutral
  Confidence: 97.5%
  Scores:     Neg=0.01 | Neu=0.97 | Pos=0.01
--------------------------------------------------

--------------------------------------------------
  Input:      I love this w ana mabsout
  Cleaned:    i love this w ana mabsout
  Prediction: ⚪ Neutral
  Confidence: 97.1%
  Scores:     Neg=0.02 | Neu=0.97 | Pos=0.01
--------------------------------------------------
```

mBERT loaded successfully onto GPU with 177M parameters, and tokenization analysis confirmed zero OOV tokens on both Arabic script and cleaned Arabizi text.

```python
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch

MODEL_NAME = "bert-base-multilingual-cased"
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

# Load with 2-class head for SOURCE domain training first
src_model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME, num_labels=2  # binary: pos/neg for mksaad
).to(device)

total_params = sum(p.numel() for p in src_model.parameters())
trainable_params = sum(p.numel() for p in src_model.parameters()
                       if p.requires_grad)

print(f"Total parameters:     {total_params:,}")
print(f"Trainable parameters: {trainable_params:,}")
print(f"Model on device:      {device}")
```